

Submitter:CFP group

Submission Date: 2021-08-22

Document: WG14 N2797

Title: N2797: *_HAS_SUBNORM==0 implies what?

Reference Documents:

Summary: A reader has asked some of the CFP group members:

For C17, if *_HAS_SUBNORM is 0, what does that imply? For example, what is `fpclassify(subnormal)`? Is it: `FP_SUBNORMAL` or `FP_ZERO`, or undefined behavior (UB) since the behavior is not defined?

Discussion

Some opinions from stackoverflow.com say (since the C standard does not explicitly define the behavior of flushed subnormals), it is undefined behavior (UB). Other opinions say treat them as another form of zero.

5.2.4.2.2, paragraph 14 defines what *_HAS_SUBNORM means in vague terms:

The presence or absence of subnormal numbers is characterized by the implementation-defined values of `FLT_HAS_SUBNORM`, `DBL_HAS_SUBNORM`, and `LDBL_HAS_SUBNORM`:

- 1 indeterminable 27)
- 0 absent (type does not support subnormal numbers)28)
- 1 present (type does support subnormal numbers)

27) Characterization as indeterminable is intended if floating-point operations do not consistently interpret subnormal representations as zero, nor as nonzero.

28) Characterization as absent is intended if no floating-point operations produce subnormal results from non-subnormal inputs, even if the type format includes representations of subnormal numbers.

Aside: Footnote 28 implies: If subnormal results cannot be produced from floating-point operations, then subnormal operands cannot happen. Creating a subnormal via union tricks does not mean subnormals are supported.

IEC 60559-2019 has in 8.2 Resuming alternate exception handling attributes

-- abruptUnderflow

When underflow is signaled because a tiny non-zero result is detected, replace the default result with a zero of the same sign or a minimum normal rounded result of the same sign, raise the underflow flag, and signal the inexact exception. When `roundTiesToEven`, `roundTiesToAway`, or the `roundTowardZero` attribute is applicable, the rounded result magnitude shall be zero. When the `roundTowardPositive` attribute is applicable, the rounded result magnitude shall be the minimum normal magnitude for positive tiny results, and zero for negative tiny results. When the `roundTowardNegative` attribute is applicable, the rounded result magnitude shall be the minimum normal magnitude for negative tiny results, and zero for positive tiny results. This attribute has no effect on the interpretation of subnormal operands.

The above has not been added to C23.

There are architectures, such as ARM (used in many cell phones), Intel x86 with SSE FPU, and NVIDIA graphics chips, that have control bits for two independent actions:

- Treat subnormal operands as zero.
- Flush subnormal results to zero.

These two aspects correspond to the Intel x86 MXCSR bits DAZ and FTZ, respectively, except that x86 detects tininess after rounding and ARM detects it before rounding, so exactly which results are flushed are different for multiply and multiply-add.

ARM standard floating-point arithmetic flushes both operands and results to zero. However, the copy (load/store), negate, and absolute value instructions never flush subnormals.

Early versions of ARM let the sign of the zeros from flushed subnormals be implementation defined. Later versions set the sign of the zero to be the sign of the subnormal. Rounding control has no effect.

IEC 60559 decimal floating-point values can have multiple representations (cohort) of the same value. In particular, the value +0.0 has a representation for every possible exponent value. Likewise, binary subnormal representations that are consistently treated as zero are just another representation of zero.

Some implementations provide extensions that allow the user's program to change flush control bits at runtime. So, `*_HAS_SUBNORM` could be a dynamic runtime property (not a constant translation time property); so could be like `FLT_ROUNDS`.

There was a proposal to redo `*_HAS_SUBNORMAL` along the lines of:

Support for subnormal numbers is characterized by the implementation-defined values of `FLT_HAS_SUBNORM`, `DBL_HAS_SUBNORM`, and `LDBL_HAS_SUBNORM`:

- 0 The type does not have subnormal numbers, or the type has subnormal numbers but arithmetic operations in the default rounding mode typically treat subnormal operands as zeros and replace subnormal results with zeros.
- 1 The type has subnormal numbers which operations consistently treat as nonzero. Appropriate for types with IEC 60559-style gradual underflow.
- 2 The type has subnormal numbers. Arithmetic operations in the default rounding mode typically replace subnormal results with zeros, but treat subnormal operands as nonzero.
- 4 The type has subnormal numbers. Arithmetic operations in the default rounding mode typically treat subnormal operands as zeros, but do not replace subnormal results with zeros.
- -1 The type has subnormal numbers but their support is not consistent in any of the above ways.

But, we decided that that still had issues.

While subnormal numbers are part of the C floating-point model, they are not required. Except for IEC 60559, we don't have a standard (official or de facto) for how they behave. Implementations that have subnormal representations have treated them in a variety of significantly different ways. The current `*_HAS_SUBNORMAL` specification is too ambiguous for implementation or use. CFP has not found a way to fix the ambiguities and accommodate the variety of implementations. CFP does not have the proper constituency, the time, or the charter to develop a specification for all implementations to the extent needed for useful characterization macros. Removing `*_HAS_SUBNORMAL` now would clear the way for proper consideration for a future C revision.

In addition, we believe that a blanket statement about flushed subnormals should be added to C23 to make it clear that C does allow flushing subnormals. They are not erroneous data, therefore, they are not undefined behavior. This would be better than adding something to each operation, macro, and function to define what happens with flushed subnormals.

So, to answer the question we were asked: For C17, if `*_HAS_SUBNORM` is 0, what is `fpclassify(subnormal)`? It is `FP_ZERO`.

Proposal

Add to 5.2.4.2.2, a new paragraph after paragraph 5:

Whether and in what cases subnormal numbers are treated as zeros is implementation defined. Subnormal numbers that in some cases are treated by arithmetic operations as zeros are properly classified as subnormal. However, object representations that could represent subnormal numbers but that are always treated by arithmetic operations as zeros are noncanonical zeros, and the values are properly classified as zero, not subnormal. IEC 60559 arithmetic (with default exception handling) always treats subnormal numbers as nonzero.

Remove 5.2.4.2.2, paragraph 14 (and its footnotes):

~~The presence or absence of subnormal numbers is characterized by the implementation-defined values of FLT_HAS_SUBNORM, DBL_HAS_SUBNORM, and LDBL_HAS_SUBNORM~~
~~-1 indeterminable²⁷⁾~~
~~0 absent (type does not support subnormal numbers)²⁸⁾~~
~~1 present (type does support subnormal numbers)~~
~~27) Characterization as indeterminable is intended if floating-point operations do not consistently interpret subnormal representations as zero, nor as nonzero.~~
~~28) Characterization as absent is intended if no floating-point operations produce subnormal results from non-subnormal inputs, even if the type format includes representations of subnormal numbers.~~

Straw poll

- Should the proposal be added to C23?